

Python 程序设计

第 5 讲 (b): Python 函数的参数

HY Deng

hydeng@shiep.edu.cn



本讲的主要内容

1 1. Python 函数的参数

- 1.1 必需参数（位置参数）
- 1.2 关键字参数
- 1.3 默认参数
- 1.4 不定长参数

2 2. 可变对象与参数陷阱

3 3. Python 函数应用示例

- 例 1: 三角形的面积
- 例 2: 利用素数验证哥德巴赫猜想
- 例 3: 判断幸运数对

什么是函数的参数：函数接口（Function Interface）

- 函数是一个“黑箱”
 - 调用者不关心函数内部如何实现
 - 只通过接口与函数交互
- 函数接口由三部分组成
 - 函数名（做什么）
 - 参数列表（需要什么信息）
 - 返回值（给出什么结果）
- 参数的作用
 - 向函数“传递外部信息”
 - 决定函数行为的具体细节

参数是函数接口中最重要、最灵活的部分

Python 函数参数的类型（概览）

1 必需参数（位置参数）

- 按位置传递
- 调用时必须提供
- 是函数最基本的参数形式

2 关键字参数

- 按“参数名 = 值”传递
- 提高代码可读性
- 调用顺序更灵活

Python 函数参数的类型（概览）

3 默认参数

- 在函数定义时给出默认值
- 调用时可以省略
- 用于描述“常见情况”

4 不定长参数

- 接受数量不确定的参数
- 提高函数的通用性和扩展性

本讲的主要内容

1 1. Python 函数的参数

- 1.1 必需参数（位置参数）
- 1.2 关键字参数
- 1.3 默认参数
- 1.4 不定长参数

2 2. 可变对象与参数陷阱

3 3. Python 函数应用示例

- 例 1: 三角形的面积
- 例 2: 利用素数验证哥德巴赫猜想
- 例 3: 判断幸运数对

位置参数（必需参数）：函数最基本的参数形式

■ 位置参数（positional parameters）

- 按参数位置进行传递
- 调用时顺序不能改变
- 调用时数量必须完全一致

位置参数（必需参数）：函数最基本的参数形式

```
def add(a, b):  
    return a + b
```

```
add(3, 5)      # 正确：a=3, b=5
```

```
# add(3)      # 错误：参数数量不足
```

```
# add(3, 5, 7) # 错误：参数数量过多
```

■ **a, b**: 形参（parameter）——函数定义阶段使用

■ **3, 5**: 实参（argument）——函数调用阶段提供

位置参数 = 最严格、最基础的函数接口形式

本讲的主要内容

1 1. Python 函数的参数

- 1.1 必需参数（位置参数）
- 1.2 关键字参数
- 1.3 默认参数
- 1.4 不定长参数

2 2. 可变对象与参数陷阱

3 3. Python 函数应用示例

- 例 1: 三角形的面积
- 例 2: 利用素数验证哥德巴赫猜想
- 例 3: 判断幸运数对

位置参数的局限：顺序一错，含义就错

- 位置参数依赖“顺序”
- 参数一多，可读性迅速下降
- 调用者必须记住每个参数的位置含义

位置参数的局限：顺序一错，含义就错

```
def move(x, y, vx, vy):  
    x += vx  
    y += vy  
    return x, y
```

```
move(0, 0, 1, -1)    # 正确，但语义不直观  
move(0, 0, -1, 1)   # 顺序不变，但含义已变
```

问题：参数“看不懂”，容易用错

关键字参数 (Keyword Arguments)

- 调用函数时使用 参数名 = 值
- 参数通过“名字”而非“位置”进行匹配
- 调用顺序不再重要

```
move(x=0, y=0, vx=1, vy=-1)  
move(vx=1, vy=-1, x=0, y=0) # 顺序可以随意
```

- 优点
 - 语义清晰
 - 可读性强
 - 不容易传错参数

位置参数 vs 关键字参数

位置参数	关键字参数
按顺序传参	按名称传参
顺序不能错	顺序可任意
参数多时不直观	参数语义清晰
适合参数少的函数	适合参数多或含义复杂的函数

关键字参数 = 更“人类友好”的函数调用方式

关键字参数: 示例 1

```
def area(width, height):  
    return width * height
```

#调用函数

```
S1 = area(width=4, height=5) #调用方式1
```

```
S2 = area(height=5, width=4) #调用方式2
```

```
print(f"area1 = {S1}")
```

```
print(f"area2 = {S2}")
```

关键字参数: 示例 2

```
def power(base, exp):  
    return base ** exp
```

```
power(base=2, exp=3)
```

```
power(exp=3, base=2)
```

- 顺序不重要
- 可读性极强

使用关键字参数调用：示例 3

```
connect(  
    ip="192.168.1.1",  
    port=8080,  
    timeout=5,  
    retry=3  
)
```

- 参数含义一目了然
- 顺序不再重要

真实世界的 Python

- matplotlib
- pandas
- requests

几乎所有成熟库都大量使用关键字参数

本讲的主要内容

1 1. Python 函数的参数

- 1.1 必需参数 (位置参数)
- 1.2 关键字参数
- 1.3 默认参数
- 1.4 不定长参数

2 2. 可变对象与参数陷阱

3 3. Python 函数应用示例

- 例 1: 三角形的面积
- 例 2: 利用素数验证哥德巴赫猜想
- 例 3: 判断幸运数对

默认参数: 让函数“更好用”的接口设计

- **默认参数**: 在函数定义时为参数指定默认值
- 调用函数时可以选择是否显式提供该参数
- 用于描述“最常见”“最合理”的默认行为

```
def power(x, n=2):  
    return x ** n
```

```
power(3)          # 使用默认参数 n=2
```

```
power(3, 3)       # 显式传入 n=3
```

默认参数 = 降低调用成本, 提高可用性

默认参数与关键字参数的配合

- 默认参数通常与关键字参数一起使用
- 调用者只关心“想改的那一项”，其余参数保持默认
- 好的默认值 = 好的函数接口

```
def move(x, y, vx=0, vy=0):  
    return x + vx, y + vy
```

<code>move(1, 2)</code>	# 不移动
<code>move(1, 2, vx=1)</code>	# 只在 x 方向移动
<code>move(1, 2, vy=-1)</code>	# 只在 y 方向移动
<code>move(1, 2, vx=1, vy=-1)</code>	# 同时移动

默认参数的规则与注意事项

■ 规则 1:

- 默认参数必须放在非默认参数之后

■ 规则 2:

- 默认值在函数定义时绑定
- 而不是在调用时生成

默认参数是“接口设计问题”，不只是语法问题

默认参数的硬性规则

默认参数必须放在最后

■ 正确:

$$f(a, b = 1)$$

■ 错误:

$$f(a = 1, b)$$

为什么默认参数必须在后？

- Python 按位置匹配参数
- 一旦省略：
 - 后面的就无法判断

接口不能产生歧义

错误示例: 混用顺序错误

```
def f(a, b, c):  
    pass
```

```
f(a=1, 2, 3)    # 错误
```

语法错误

默认参数: 示例 1

```
def greet(name, msg="Welcome"):  
    return name + ", " + msg  
  
greet("Alice")  
greet("Bob", "Good morning")
```

- msg 有默认值
- 可以省略

默认参数: 示例 2

```
def area( width, height = 1 ):
    return width * height
```

#调用函数

```
S1 = area(width=4) #调用方式1:默认参数
```

```
S2 = area(width=4, height=5) #调用方式2
```

```
print(f"area1 = {S1}")
```

```
print(f"area2 = {S2}")
```

关键字参数 + 默认值: 示例 3

```
def connect(ip, port=80, timeout=10, retry=3):  
    pass  
  
connect(ip="10.0.0.1")  
connect(ip="10.0.0.1", timeout=2)
```

- 只关心想改的参数
- 其余使用默认值

本讲的主要内容

1 1. Python 函数的参数

- 1.1 必需参数（位置参数）
- 1.2 关键字参数
- 1.3 默认参数
- 1.4 不定长参数

2 2. 可变对象与参数陷阱

3 3. Python 函数应用示例

- 例 1: 三角形的面积
- 例 2: 利用素数验证哥德巴赫猜想
- 例 3: 判断幸运数对

为什么需要不定长参数?

- 有些函数无法提前确定参数个数
- 参数数量依赖 “使用场景”
- 用多个位置参数会导致接口僵硬

```
def add(a, b, c, d):  
    # 问题: 只能加 4 个数  
    return a + b + c + d
```

问题: 参数个数变化时, 接口就崩溃

不定长位置参数: *args

- *args 用于接收数量不确定的位置参数
- 所有多余的位置参数会被打包成一个 元组
- args 只是名字，* 才是关键

```
def add_all(*args):  
    print(args)  
    return sum(args)
```

```
add_all(1, 2)
```

```
add_all(1, 2, 3, 4, 5)
```

args 的类型是 tuple

如何使用 *args?

- 可以像普通序列一样操作
- 常见操作: 遍历、求和、取长度

```
def average(*args):  
    if len(args) == 0:  
        return None  
    return sum(args) / len(args)
```

```
average(10, 20)
```

```
average(1, 2, 3, 4)
```

**args = 灵活但需要自己检查合法性*

*args 的设计意义

■ 优点

- 接口灵活
- 适用于参数数量变化大的场景

■ 代价

- 参数语义不够明确
- 需要函数内部自行做检查

■ 设计建议

- 适合“同类数据的集合”
- 不适合语义完全不同的参数

***args 是“灵活性”换来的**

*args: 示例

```
def sum_variable(arg1, *var_tuple ):
    "求从第1位置开始的列表之和"
    print (f"arg1 = {arg1}")
    print (f"var_tuple = {var_tuple}")
    print (f"var_tuple的类型为: {type(var_tuple)}")
    return sum(var_tuple)

# 调用sum_variable函数
result1 = sum_variable('不定长参数', 200, 10, 1)
print(f"result1 = {result1}")

print("—" * 20)
result2 = sum_variable(200, 10, 1)
print(f"result2 = {result2}")
```

*args: 示例（续）

输出结果如下：

```
arg1 = 不定长参数
var_tuple = (200, 10, 1)
var_tuple的类型为: <class 'tuple'>
result1 = 211

-----

arg1 = 200
var_tuple = (10, 1)
var_tuple的类型为: <class 'tuple'>
result2 = 11
```

如果在函数调用时没有指定参数，它就是一个空元组

为什么需要 **kwargs?

- 有些参数不是“数量不确定”，而是“种类不确定”
- 参数是否出现，取决于使用者
- 传统位置参数无法表达“可选配置项”

接口需要支持“配置型参数”

关键字不定长参数: ****kwargs**

- ****kwargs** 接收任意数量的关键字参数
- 所有参数会被打包成一个字典 **dict**
- 键是参数名, 值是参数值

```
def show_info(**kwargs):  
    print(kwargs)  
  
show_info(name="Alice", age=20)  
show_info(debug=True, timeout=10)
```

kwargs 的类型是 **dict**

如何使用 `**kwargs`?

- 像普通字典一样使用
- 常见操作: 取值、判断是否存在

```
def connect(**kwargs):  
    host = kwargs.get("host", "localhost")  
    port = kwargs.get("port", 3306)  
    debug = kwargs.get("debug", False)  
    print(host, port, debug)  
  
connect(host="127.0.0.1")  
connect(host="127.0.0.1", port=8000, debug=True)
```

`**kwargs` = 可选参数的集合

*args 与 **kwargs

*args

接收多余的位置参数

元组（tuple）

顺序重要

适合同类数据

**kwargs

接收多余的关键字参数

字典（dict）

顺序无关

适合配置项

两者解决的是“不同维度”的灵活性

`**kwargs` 的工程意义

- 提供向后兼容的接口扩展能力
- 新增功能无需修改函数签名
- 广泛用于框架、库、底层接口

`**kwargs` = 为未来预留接口

可变关键字参数: 示例 1

```
def greet(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key} : {value}")  
  
greet(name="Alice", age=25, city="Beijing")
```

输出: name : Alice, age : 25, city : Beijing

不同参数类型混合使用：示例 2

```
def func(a, b, *args, **kwargs):  
    print(a, b)  
    print("args:", args)  
    print("kwargs:", kwargs)  
  
func(1, 2, 3, 4, x=10, y=20)
```

输出: a=1, b=2, args=(3,4), kwargs='x':10,'y':20

不同参数类型混合使用：示例 3

```
1  def multi_parameters(arg1, arg2='Default', *args, **kwargs):
5      print( 'tuple arg:', each_arg)
6      for each_kwarg in kwargs.keys():
7          print( 'dict arg', each_kwarg, ': ', kwargs[each_kwarg])
8  #调用函数
9  multi_parameters('Python', 'Excellent', 'I', 'Love' \
10                 language='Python', number=100, time='Years')
```

函数参数顺序规则

- 1 必选参数
- 2 默认参数
- 3 *args
- 4 **kwargs

严格遵守顺序，否则会报错！

应用示例 1: 日志函数

```
def log(message, *args, **kwargs):  
    print("[LOG]", message)  
    if args:  
        print("args:", args)  
    if kwargs:  
        print("kwargs:", kwargs)  
  
log("Start", 1,2, user="Alice", status="ok")
```

应用示例 2: 通用求和器

```
def calc_sum(*nums, scale=1):  
    total = sum(nums)  
    return total * scale  
  
print(calc_sum(1,2,3))  
print(calc_sum(1,2,3, scale=10))
```

小结 1

- *args 用于收集任意位置参数 → tuple
- **kwargs 用于收集任意关键字参数 → dict
- 混合使用时顺序固定：必选 → 默认 → *args
→ **kwargs
- 工程中非常常用：扩展接口、解耦函数设计

目标：能写任意灵活函数接口

为什么要区分可变对象与不可变对象？

- Python 中一切皆对象
- 对象是否“可变”，直接影响函数参数行为
- 很多隐蔽 **Bug** 并非语法错误，而是对象语义错误

参数传递的本质 = 对象的传递

不可变对象 (Immutable Objects)

- 对象创建后，其内容不能被修改
- 修改操作会创建新对象

```
x = 10
y = x
x += 1
print(x)    # 11
print(y)    # 10
```

常见不可变对象:

- int, float
- str
- tuple

可变对象（Mutable Objects）

- 对象创建后，其内容可以被原地修改
- 多个变量可能指向同一个对象

```
lst1 = [1, 2, 3]
lst2 = lst1
lst1.append(4)
print(lst1)    # [1, 2, 3, 4]
print(lst2)    # [1, 2, 3, 4]
```

常见可变对象：

- list
- dict
- set

参数传递中的经典陷阱

看起来像“值传递”，实际是“对象共享”

```
def add_item(lst):  
    lst.append(100)  
  
data = [1, 2, 3]  
add_item(data)  
print(data)
```

- 函数内部修改了参数
- 函数外部对象被悄悄改变
- 初学者最容易误判

工程经验：使用拷贝避免副作用

- 函数不应“偷偷修改”调用者的数据
- 可变对象应显式拷贝后再操作

```
def safe_add_item(lst):  
    new_lst = lst.copy()    # 硬拷贝  
    new_lst.append(100)  
    return new_lst  
  
data = [1, 2, 3]  
result = safe_add_item(data)
```

好函数：无副作用或副作用可控

参数传递的工程原则总结

- 不可变对象：放心传递
- 可变对象：谨慎修改
- 修改输入数据前要三思
- 必要时使用 `copy` / `deepcopy`

Python 不是值传递，也不是引用传递，而是对象传递

示例 1: 不可变对象的表现

```
def inc(x):  
    x = x + 1  
  
a = 10  
inc(a)  
print(a)
```

输出: 10

示例 2: 可变对象的表现

```
def inc_list(lst):  
    lst.append(1)  
  
a = [10]  
inc_list(a)  
print(a)
```

输出: [10, 1]

示例 3: 默认参数的安全写法

```
def append_item(x, lst=None):  
    if lst is None:  
        lst = []  
    lst.append(x)  
    return lst
```

官方推荐写法

小结 2

- 参数传递的是对象引用
- 可变对象会产生副作用
- 默认参数不能用可变对象

本讲的主要内容

1 1. Python 函数的参数

- 1.1 必需参数（位置参数）
- 1.2 关键字参数
- 1.3 默认参数
- 1.4 不定长参数

2 2. 可变对象与参数陷阱

3 3. Python 函数应用示例

- 例 1: 三角形的面积
- 例 2: 利用素数验证哥德巴赫猜想
- 例 3: 判断幸运数对

函数应用示例 1

例 1: 三角形的面积

已知三角形的三条边长 a, b, c , 根据下面的公式计算三角形的面积.

$$S = \sqrt{p(p-a)(p-b)(p-c)}$$

其中, $p = \frac{a+b+c}{2}$.

函数应用示例 1

例 1: 三角形的面积

```
1  def triangle_area(a, b, c):  
2      p = (a + b + c) / 2  
3      if ((a+b) > c) & ((b+c)>a) &((c+a)>b):  
4          return (p*(p-a)*(p-b)*(p-c)) ** 0.5  
5      else:  
6          return -1 #-1标记为不能构成三角形  
7  
8  a, b, c = 3, 4, 5  
9  S = triangle_area(a, b, c)  
10 print(f'三角形的三条边为{a}, {b}, {c}, 其面积为{S}')
```

本讲的主要内容

1 1. Python 函数的参数

- 1.1 必需参数（位置参数）
- 1.2 关键字参数
- 1.3 默认参数
- 1.4 不定长参数

2 2. 可变对象与参数陷阱

3 3. Python 函数应用示例

- 例 1: 三角形的面积
- 例 2: 利用素数验证哥德巴赫猜想
- 例 3: 判断幸运数对

函数应用示例 2

例 2: 利用素数验证哥德巴赫猜想

写一个自定义函数，用于判断一个正整数是否为素数，并利用该函数验证哥德巴赫猜想，即任意大于等于 4 的偶数都可以分解为两个素数之和，要求输出测试数据的所有种组合。

函数应用示例 2

例 2: 利用素数验证哥德巴赫猜想

```
1  # 判断素数的函数
2  def is_prime(n):
3      if n == 1:
4          return False
5      for i in range(2, n):
6          if n % i == 0 :
7              return False
8      return True
9
10 y = int(input("输入大于4的偶数"))
11 for i in range(2, y // 2):
12     if is_prime(i) and is_prime(y-i):
13         print(i, y-i)
```

本讲的主要内容

1 1. Python 函数的参数

- 1.1 必需参数（位置参数）
- 1.2 关键字参数
- 1.3 默认参数
- 1.4 不定长参数

2 2. 可变对象与参数陷阱

3 3. Python 函数应用示例

- 例 1: 三角形的面积
- 例 2: 利用素数验证哥德巴赫猜想
- 例 3: 判断幸运数对

函数应用示例 3

例 3: 判断幸运数对

写一个自定义函数，用于判断两个数是否为幸运数对。所谓幸运数对是指两数相差 3，且每数的各位数字之和能被 6 整除的一对数，如 147 和 150 就是幸运数对。要求找出所有的三位数幸运数对。

函数应用示例 3: codes

```
def digit_sum(n):  
    """计算整数 n 的各位数字之和"""  
    return sum(int(d) for d in str(n))  
  
def is_lucky_pair(a, b):  
    """  
    判断 a 和 b 是否为幸运数对  
    条件:  
    1. 两数相差 3  
    2. 各位数字之和能被 6 整除  
    """  
    if (abs(a - b) == 3) and (digit_sum(a) % 6 == 0) and (  
        digit_sum(b) % 6 == 0):  
        return True  
    else:  
        return False
```

函数应用示例 3: codes-cont.

```
# 查找所有三位数幸运数对
lucky_pairs = []
for a in range(100, 1000):
    b = a + 3
    if b < 1000 and is_lucky_pair(a, b):
        lucky_pairs.append((a, b))
# 输出结果
for pair in lucky_pairs:
    print(pair)

print(f"共找到 {len(lucky_pairs)} 对三位数幸运数对")
```

函数应用示例 3: 输出结果

(129, 132)

(138, 141)

(147, 150)

(189, 192)

(219, 222)

(228, 231)

(237, 240)

(279, 282)

(288, 291)

(309, 312)

(318, 321)

(327, 330)

(369, 372)

(378, 381)

函数应用示例 3: 输出结果

(387, 390)

(408, 411)

(417, 420)

(459, 462)

(468, 471)

(477, 480)

(507, 510)

(549, 552)

(558, 561)

(567, 570)

(639, 642)

(648, 651)

(657, 660)

(729, 732)

(738, 741)

函数应用示例 3: 输出结果

(747, 750)

(789, 792)

(819, 822)

(828, 831)

(837, 840)

(879, 882)

(888, 891)

(909, 912)

(918, 921)

(927, 930)

(969, 972)

(978, 981)

(987, 990)

共找到 42 对三位数幸运数对

1. Python 函数的参数
2. 可变对象与参数陷阱
3. Python 函数应用示例

例 1: 三角形的面积

例 2: 利用素数验证哥德巴赫猜想

例 3: 判断幸运数对

Thanks!